



NexusWealth - Audit Security Assessment

CertiK Assessed on Oct 15th, 2025





CertiK Assessed on Oct 15th, 2025

NexusWealth - Audit

The security assessment was prepared by CertiK.

Executive Summary

TYPES

ERC-20

ECOSYSTEM

EVM Compatible

METHODS

Formal Verification, Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 10/10/2025

Final report published on 10/17/2025

Vulnerability Summary



15

Total Findings

11

Resolved

0

Partially Resolved

4

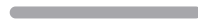
Acknowledged

0

Declined

2 Centralization

2 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

1 Major

1 Resolved



Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

5 Medium

4 Resolved, 1 Acknowledged



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

5 Minor

4 Resolved, 1 Acknowledged



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

2 Informational

2 Resolved



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | NEXUSWEALTH - AUDIT

Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Findings

[NEA-02 : Initial Token Distribution](#)

[NEA-03 : Centralization Risks](#)

[NEA-04 : Missing Token Refund In Bridge Cancellation](#)

[NEA-05 : Bridge Functionality Failure](#)

[NEA-06 : Empty Proposals Without Execution Logic](#)

[NEA-07 : Succeeded Proposal Can Be Cancelled](#)

[NEA-08 : Cross-Chain Token Bridging Enables Governance Manipulation](#)

[NEA-09 : Inaccurate Mint Tracking And Supply Cap Validation](#)

[NEA-10 : `totalBurned` Tracking Inconsistency](#)

[NEA-11 : Manipulatable `quorum`](#)

[NEA-12 : Same-Block Snapshot Lets Proposer Manipulate Voting Power](#)

[NEA-13 : Missing Blacklist Check For Bridge Recipient](#)

[NEA-14 : Token `maxSupply` Restriction Is Ineffective On L2 Chains](#)

[NEA-15 : Excess ETH Over The Fixed BridgeFee Is Silently Taken And Accounted As Fee](#)

[NEA-16 : Missing Zero Address Check In `initiateBridge`](#)

Appendix

Disclaimer

CODEBASE | NEXUSWEALTH - AUDIT

Repository

<https://github.com/NexusWealth/nwis-token/blob/3622cb0024a10363f2d5492a2dcc8f2464f6c041/NexusWealthToken.sol>

Commit

[3622cb0024a10363f2d5492a2dcc8f2464f6c041](#)

AUDIT SCOPE | NEXUSWEALTH - AUDIT

NexusWealth/nwis-token



NexusWealthToken.sol

APPROACH & METHODS | NEXUSWEALTH - AUDIT

This audit was conducted for NexusWealth to evaluate the security and correctness of the smart contracts associated with the NexusWealth - Audit project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Formal Verification, Manual Review, and Static Analysis.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

FINDINGS | NEXUSWEALTH - AUDIT



15
Total Findings

0
Critical

2
Centralization

1
Major

5
Medium

5
Minor

2
Informational

This report has been prepared for NexusWealth to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 15 issues were identified. Leveraging a combination of Formal Verification, Manual Review & Static Analysis the following findings were uncovered:

ID	Title	Category	Severity	Status
NEA-02	Initial Token Distribution	Centralization	Centralization	● Acknowledged
NEA-03	Centralization Risks	Centralization	Centralization	● Acknowledged
NEA-04	Missing Token Refund In Bridge Cancellation	Logical Issue	Major	● Resolved
NEA-05	Bridge Functionality Failure	Logical Issue	Medium	● Resolved
NEA-06	Empty Proposals Without Execution Logic	Logical Issue	Medium	● Resolved
NEA-07	Succeeded Proposal Can Be Cancelled	Logical Issue	Medium	● Acknowledged
NEA-08	Cross-Chain Token Bridging Enables Governance Manipulation	Design Issue	Medium	● Resolved
NEA-09	Inaccurate Mint Tracking And Supply Cap Validation	Logical Issue	Medium	● Resolved
NEA-10	<code>totalBurned</code> Tracking Inconsistency	Inconsistency	Minor	● Resolved
NEA-11	Manipulatable <code>quorum</code>	Inconsistency	Minor	● Acknowledged
NEA-12	Same-Block Snapshot Lets Proposer Manipulate Voting Power	Logical Issue	Minor	● Resolved

ID	Title	Category	Severity	Status
NEA-13	Missing Blacklist Check For Bridge Recipient	Logical Issue	Minor	● Resolved
NEA-14	Token <code>maxSupply</code> Restriction Is Ineffective On L2 Chains	Inconsistency	Minor	● Resolved
NEA-15	Excess ETH Over The Fixed BridgeFee Is Silently Taken And Accounted As Fee	Inconsistency	Informational	● Resolved
NEA-16	Missing Zero Address Check In <code>initiateBridge</code>	Inconsistency	Informational	● Resolved

NEA-02 | Initial Token Distribution

Category	Severity	Location	Status
Centralization	● Centralization	NexusWealthToken.sol (v1): 108	● Acknowledged

Description

Initial supply tokens are sent to the contract deployer. This is a centralization risk because the deployer can distribute tokens without obtaining the consensus of the community. Any compromise to these addresses may allow a hacker to steal and sell tokens on the market, resulting in severe damage to the project.

Recommendation

It is recommended that the team be transparent regarding the initial token distribution process. The token distribution plan should be published in a public location that the community can access. The team should make efforts to restrict access to the private keys of the deployer account or EOAs. A multi-signature ($\frac{2}{3}$, $\frac{3}{5}$) wallet can be used to prevent a single point of failure due to a private key compromise. Additionally, the team can lock up a portion of tokens, release them with a vesting schedule for long-term success, and deanonymize the project team with a third-party KYC provider to create greater accountability.

Alleviation

[NexusWealth, 10/15/2025]: The initial supply will be transferred to the Presale contract by the CEO.

[CertiK, 10/15/2025]: It is suggested to implement the aforementioned methods to avoid centralized failure. Also, CertiK strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

NEA-03 | Centralization Risks

Category	Severity	Location	Status
Centralization	● Centralization		● Acknowledged

Description

In the contract `NexusWealthToken` the role `_owner` has authority over the functions below. Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and

- `setBridgeOperator()` : Sets or removes an address as a bridge operator.
- `setBridgeFee()` : Updates the fee required for bridge transactions.
- `setBlacklistOperator()` : Sets or removes an address as a blacklist operator.
- `setSupportedChain()` : Adds or removes chain IDs from the list of supported chains for bridging.
- `pause()` : Pauses all token transfers, minting, and burning operations.
- `unpause()` : Resumes all token operations after being paused.
- `mint()` : Creates new tokens and assigns them to an address, increasing total supply.
- `withdrawBridgeFees()` : Withdraws collected bridge fees to a specified address.
- `setVotingParams()` : Configures governance parameters including proposal threshold, voting delay, period, and quorum.
- `setBlacklistStatus()` : Adds or removes an address from the blacklist, preventing transfers to/from that address.
- `cancelBridge()` : Cancels a pending bridge request, can only be called by the request initiator or the owner.
- `cancel()` : Cancels a governance proposal, can only be called by the proposer or the owner.

In the contract `NexusWealthToken` the role `blacklistOperators` has authority over the functions below. Any compromise to the `blacklistOperators` account may allow the hacker to take advantage of this authority and

- `setBlacklistStatus()` : Adds or removes an address from the blacklist, preventing transfers to/from that address.

In the contract `NexusWealthToken` the role `bridgeOperators` has authority over the functions below. Any compromise to the `bridgeOperators` account may allow the hacker to take advantage of this authority and

- `processBridge()` : Processes a bridge request by minting tokens to the destination address.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts

with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign ($\frac{2}{3}$, $\frac{3}{5}$) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

Alleviation

[NexusWealth, 10/15/2025]: The Ownership will be transferred to a Gnosis safe account.

[Certik, 10/15/2025]: It is suggested to implement the aforementioned methods to avoid centralized failure. Also, Certik strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

NEA-04 | Missing Token Refund In Bridge Cancellation

Category	Severity	Location	Status
Logical Issue	● Major	NexusWealthToken.sol (v1): 181	● Resolved

Description

The bridge functionality in `NexusWealthToken` allows users to initiate cross-chain transfers by burning tokens on the source chain. When a user calls `initiateBridge()`, the contract burns the specified amount of tokens from their balance and creates a bridge request.

Before the bridge operator processes the request, either the request initiator or the contract owner can cancel the request via `cancelBridge()`. However, the cancellation function only updates the request status to canceled but does not refund the burned tokens back to the user's balance.

The root cause is in the `cancelBridge()` function only sets `r.canceled = true` without performing any token minting operation to return the previously burned tokens.

This results in direct loss of funds for users who cancel bridge requests.

Proof of Concept

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {Test, console} from "forge-std/Test.sol";
import {NexusWealthToken, Pausable} from "../src/NexusWealthToken.sol";

contract NexusWealthTokenGTest is Test {
    NexusWealthToken public token;
    address public owner;
    address public user1;
    address public user2;
    address public bridgeOperator;
    address public blacklistOperator;
    uint256 public constant MAX_SUPPLY = 1_000_000_000;
    uint256 public constant INITIAL_SUPPLY = 100_000_000;
    uint8 public constant DECIMALS = 18;

    function setUp() public {
        owner = makeAddr("owner");
        user1 = makeAddr("user1");
        user2 = makeAddr("user2");
        bridgeOperator = makeAddr("bridgeOperator");
        blacklistOperator = makeAddr("blacklistOperator");

        vm.prank(owner);
        token = new NexusWealthToken(
            "NexusWealthToken",
            "NWT",
            DECIMALS,
            MAX_SUPPLY,
            INITIAL_SUPPLY
        );

        vm.prank(owner);
        token.setBridgeOperator(bridgeOperator, true);
        vm.prank(owner);
        token.setBlacklistOperator(blacklistOperator, true);
        vm.prank(owner);
        token.setSupportedChain(1, true);
    }

    function test_NoRefundOnCancel() public {
        uint256 bridgeAmount = 5000 * (10 ** DECIMALS);
        uint256 initialOwnerBalance = token.balanceOf(owner);
        uint256 initialTotalSupply = token.totalSupply();

        vm.prank(owner);
        token.initiateBridge{value: 0}(user2, bridgeAmount, 1);
    }
}
```

```
        assertEq(token.balanceOf(owner), initialOwnerBalance - bridgeAmount, "Owner  
balance should decrease after initiation");  
        assertEq(token.totalSupply(), initialTotalSupply - bridgeAmount, "Total  
supply should decrease after initiation");  
  
        uint256 requestId = token.bridgeRequestCount();  
  
        // Cancel the bridge request.  
        vm.prank(owner);  
        token.cancelBridge(requestId);  
  
        // Assert that the tokens were NOT refunded.  
        // The user's balance and the total supply remain reduced.  
        assertEq(token.balanceOf(owner), initialOwnerBalance - bridgeAmount, "Owner  
balance is NOT refunded after cancel");  
        assertEq(token.totalSupply(), initialTotalSupply - bridgeAmount, "Total  
supply is NOT restored after cancel");  
  
        (,,,,, bool canceled) = token.bridgeRequests(requestId);  
        assertTrue(canceled, "Request should be marked as canceled");  
    }  
}
```

Recommendation

It is recommended to modify the `cancelBridge()` function to mint back the burned tokens (and paid fees) to the user's address when a bridge request is canceled.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue by removing the bridge functionality in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-05 | Bridge Functionality Failure

Category	Severity	Location	Status
Logical Issue	● Medium	NexusWealthToken.sol (v1): 154, 170	● Resolved

Description

The bridge functionality in `NexusWealthToken` is intended to enable cross-chain token transfers by burning tokens on a source chain and minting equivalent amounts on a target chain. The process involves two key functions:

`initiateBridge()` on the source chain and `processBridge()` on the target chain.

In the `initiateBridge()` function, users specify `to` and `targetChainId` parameters to indicate the destination chain for their bridge request. However, these crucial information are not emitted in the `BridgeInitiated` event, which only includes the `requestId`, `from` address, `amount`, and `fee`. Off-chain validators or relayers that monitor these events cannot determine the intended destination for processing.

Furthermore, if `processBridge()` is intended to process bridge requests on the target chain, while the bridge request is stored in the `bridgeRequests` mapping on the source chain, it cannot be auto-synchronized to the target chain. When bridge operators attempt to call `processBridge()` on the target chain with the same `requestId`, the target chain has no record of the request in its local `bridgeRequests` mapping. If `processBridge()` is intended to mark bridge requests as processed on the source chain, `_mint()` tokens to `r.to` address leads to a double-spending issue.

These issues result in complete failure of cross-chain bridge functionality.

Recommendation

Consider revisiting the cross-chain design, focusing on these fundamental operational steps:

- `initiateBridge()` should emit a `BridgeInitiated` that includes target chain ID and the recipient address.
- `processBridge()` should be split:
 - The operator marks the bridge request as `processed` on the source chain.
 - The operator mints the equivalent tokens for token recipient on the target chain.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue by removing the bridge functionality in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-06 | Empty Proposals Without Execution Logic

Category	Severity	Location	Status
Logical Issue	● Medium	NexusWealthToken.sol (v1): 215	● Resolved

Description

The governance functionality in `NexusWealthToken` is intended to allow token holders to propose and vote on changes to the contract. However, the current implementation of the proposal system is ineffective.

In the `createProposal()` function, proposers can only provide a description string parameter. The function creates a new Proposal struct that stores basic metadata (proposer address, description, voting timeline, vote counts) but contains no executable code or action definitions.

When a proposal passes voting and reaches the `execute()` function, the function only marks the proposal as executed and emits an event. No actual actions are performed because the proposal contains no action data. The Proposal struct lacks fields for defining actions, targets, values, or calldata that would be needed to execute meaningful changes.

Recommendation

The governance system needs to be enhanced with executable proposal actions.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue by adding the calldata and target for proposals in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-07 | Succeeded Proposal Can Be Cancelled

Category	Severity	Location	Status
Logical Issue	● Medium	NexusWealthToken.sol (v1): 255	● Acknowledged

Description

The `cancel(uint256 id)` function lets the proposer (or the owner) set `p.canceled = true` at any time as long as the proposal is not already executed or canceled. There is no check on the proposal lifecycle (e.g., not restricting to Pending/Active) or on the voting outcome. As a result, the proposer can cancel even after the voting period has ended and the proposal has achieved quorum and a majority (i.e., it has "succeeded"). Once canceled, the proposal becomes permanently finalized and cannot be executed, giving the proposer an unconditional post-vote veto over their own proposal.

Recommendation

It is recommended to add restriction that only Pending or Active proposal can be cancelled.

Alleviation

[NexusWealth, 10/17/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement, providing the following statement: "Issue acknowledged. I will fix the issue in the future, which will not be included in this audit engagement."

NEA-08 | Cross-Chain Token Bridging Enables Governance Manipulation

Category	Severity	Location	Status
Design Issue	● Medium	NexusWealthToken.sol (v1): 18	● Resolved

Description

The governance in `NexusWealthToken` uses a dynamic quorum calculation based on the current `totalSupply` :

```
200 function quorum(uint256) public view returns (uint256) {  
201     return (totalSupply() * quorumNumerator) / QUORUM_DENOMINATOR;  
202 }
```

Combined with the bridge functionality, this creates a vulnerability:

1. When users bridge tokens from Chain A to Chain B, tokens are burned on Chain A and minted on Chain B
2. This reduces `totalSupply` on Chain A and increases it on Chain B
3. Since quorum is calculated as a percentage of `totalSupply`, the required quorum changes on both chains
4. Combine this with the fact that the token circulation supply in the new chain is relatively low. An attacker can consolidate all tokens on a single new chain to manipulate governance outcomes.

The root cause is that the governance mechanism was designed without considering cross-chain supply dynamics. Each chain's governance operates independently with its own `totalSupply` calculation and token circulation supply in an individual chain.

Recommendation

Consider disabling governance mechanisms on all L2 chains, keeping governance only on the main chain.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-09 | Inaccurate Mint Tracking And Supply Cap Validation

Category	Severity	Location	Status
Logical Issue	● Medium	NexusWealthToken.sol (v1): 148, 177	● Resolved

Description

`totalMinted` is intended to track tokens that were directly minted, representing the actual circulating token supply.

However, in the `processBridge()` function, `totalMinted` is incorrectly updated when tokens are minted through bridging. This causes `totalMinted` to include both direct mints and bridge mints, making it impossible to accurately track how many tokens were actually minted by the contract owner.

The `mint()` function validates minting against the supply cap using:

```
require(totalSupply() + amount <= maxSupply, "Exceeds max supply");
```

However, `totalSupply()` does not accurately reflect the actual token supply across chains because: When tokens are bridged from Chain A to Chain B, they are burned on Chain A, reducing its `totalSupply()`. This allows more tokens to be minted on Chain A since `totalSupply()` is now lower.

Recommendation

1. Remove the incorrect `totalMinted` update from `processBridge()`
2. Modify the `mint()` function to validate against actual minted tokens instead of `totalSupply()`

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-10 | `totalBurned` Tracking Inconsistency

Category	Severity	Location	Status
Inconsistency	Minor	NexusWealthToken.sol (v1): 161	Resolved

Description

The `NexusWealthToken` contract's `totalBurned` variable is only updated within the `initiateBridge` function. The variable fails to account for tokens burned directly by users via the inherited `ERC20Burnable.burn` function, resulting in an inaccurate record of the actual total tokens burned.

Recommendation

- If the variable's true purpose is to track bridged tokens, consider renaming `totalBurned` to a name that accurately reflects this function.
- If the variable is intended to track all burned tokens, consider overriding the inherited `burn` function to ensure it updates `totalBurned` and removing the update operation from `initiateBridge()`.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue by tracking all burned token amount in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-11 | Manipulatable quorum

Category	Severity	Location	Status
Inconsistency	● Minor	NexusWealthToken.sol (v1): 200	● Acknowledged

Description

The `quorum(uint256)` in the contract is computed from the current `totalSupply` (ignoring its block number argument). The effective quorum at execution time is manipulable via supply changes after voting has finished. Any holder can reduce current `totalSupply` permissionlessly (`ERC20Burnable.burn()` and `initiateBridge()` both burn), so the `execute()` check “turnout >= quorum(..)” can be satisfied even when turnout was below the intended quorum at the proposal's snapshot.

Recommendation

Consider getting the total supply from the snapshot `getPastTotalSupply` instead.

Alleviation

[NexusWealth, 10/17/2025]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement, providing the following statement: "Issue acknowledged. I won't make any changes for the current version."

NEA-12 | Same-Block Snapshot Lets Proposer Manipulate Voting Power

Category	Severity	Location	Status
Logical Issue	Minor	NexusWealthToken.sol (v1): 218	Resolved

Description

`createProposal()` sets the proposal snapshot to the current block (`uint256 snap = block.number`) while proposer eligibility is checked at the previous block (`getPastVotes(msg.sender, block.number - 1)`). In `ERC20Votes`, `getPastVotes(account, B)` returns the votes at the end of block B, i.e., after all transactions in block B have executed. As a result, any token transfers or (re)delegations executed after `createProposal()` but still within the same block will be included in the recorded snapshot for that proposal.

A snapshot is meant to freeze voting power at a well-defined point, preventing post-proposal vote-power manipulation. By using the current block as the snapshot, the proposal's voting snapshot is not actually fixed at proposal creation time; it is finalized only at the end of that block. This enables a proposer to create a proposal and then, in the same block, reshape voting power (via transfers/delegations) to lock in an arbitrarily favorable snapshot. The threshold check at `block.number - 1` does not prevent this manipulation and, with the default `proposalThreshold = 0`, anyone can exploit it.

Recommendation

It is recommended to set the `snap` to `block.number - 1` in `createProposal()`.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-13 | Missing Blacklist Check For Bridge Recipient

Category	Severity	Location	Status
Logical Issue	Minor	NexusWealthToken.sol (v1): 154	Resolved

Description

The bridge functionality in `NexusWealthToken` allows users to transfer tokens across chains by burning them on the source chain and minting equivalent amounts on the target chain. However, the `initiateBridge()` function only checks if the sender is blacklisted but fails to verify if the recipient address is also blacklisted.

When `processBridge()` is called on the target chain, it attempts to mint tokens to the recipient address by calling `_mint(r.to, r.amount)`, which internally calls `_update(address(0), r.to, r.amount)`. The `_update()` function correctly enforces blacklist checks for recipient addresses with `require(!isBlacklisted[to], "Recipient blacklisted")`. If the recipient is blacklisted, the transaction reverts, and the bridge request cannot be processed.

This operational failure results in tokens being successfully burned on the source chain but subsequently not minted on the target chain.

Recommendation

It is recommended to add a blacklist check for the recipient address in the `initiateBridge()` function.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-14 | Token `maxSupply` Restriction Is Ineffective On L2 Chains

Category	Severity	Location	Status
Inconsistency	Minor	NexusWealthToken.sol (v1): 147	Resolved

Description

The `NexusWealthToken` contract implements a global supply cap through the `maxSupply` parameter that restricts minting on each individual chain.

However, the bridge functionality bypasses this protection: contract owners on each L2 chain can mint tokens up to their chain's `maxSupply` independently.

The root cause is that the bridge and minting mechanisms were designed without considering cross-chain supply coordination. Each chain operates independently with its own supply tracking, creating an inconsistency.

Recommendation

The capability for owner initiated token minting on L2 chains is unnecessary and should be removed from the contract.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-15 | Excess ETH Over The Fixed BridgeFee Is Silently Taken And Accounted As Fee

Category	Severity	Location	Status
Inconsistency	● Informational	NexusWealthToken.sol (v1): 157	● Resolved

Description

The `initiateBridge()` function requires `msg.value >= bridgeFee` but does not enforce equality or refund any excess. It records the entire `msg.value` into the `BridgeRequest` and adds it to `bridgeFeesCollected`. This is a logic flaw: users who send more than `bridgeFee` lose the difference irreversibly, and the contract's accounting treats the overpayment as fee. This can be practically triggered in a single call without any special conditions.

Recommendation

Consider updating the check condition to `msg.value == bridgeFee`.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue in commit [8b8907b17211439158ea95fb6411f2f29440db63](#)

NEA-16 | Missing Zero Address Check In `initiateBridge`

Category	Severity	Location	Status
Inconsistency	● Informational	NexusWealthToken.sol (v1): 154	● Resolved

Description

There is no check that the recipient address `to` is non-zero. A user can initiate a bridge to `address(0)`. The function will burn the tokens, collect the fee, and store a request pointing to the zero address. In any realistic bridging flow, this results in irrecoverable funds on the target chain (mint to zero address or failure to credit), permanently losing the burned tokens and the fee.

Recommendation

Considering adding a check to ensure that the `to` address cannot be set to zero.

Alleviation

[NexusWealth, 10/15/2025]: The team heeded the advice and resolved the issue in commit [8b8907b17211439158ea95fb6411f29440db63](#)

APPENDIX | NEXUSWEALTH - AUDIT

Finding Categories

Categories	Description
Inconsistency	Inconsistency findings refer to different parts of code that are not consistent or code that does not behave according to its specification.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

